

Physical AIの実運用

学習した方策を実機で正しく動かす



松尾・岩澤研究室

MATSUO-IWASAWA LAB UTOKYO

Deep Learning for
Action Generation

上條 達也

東京大学大学院 工学系研究科 松尾・岩澤研究室 修士 2 年

経歴

- **2024年3月** 東京大学 工学部 機械工学科 卒業
- **2024年10月～** 同 大学院 工学系研究科 技術経営戦略学専攻

インターン

AIRoA

OMRON SINIC X

研究内容 ・ ロボット学習 / 器用な操作 (DEXTEROUS MANIPULATION)

- **可変力制御** の模倣学習
- **触覚** に基づく器用な物体操作学習
- 人間の動作動画からの **ロボット方策学習**



	カリキュラム
第1回	Physical AIの概要
第2回	Physical AIの活用
第3回	Physical AIの学習（模倣学習）
第4回	Physical AIの学習（強化学習）
第5回	スケーリングとアーキテクチャ
第6回	実世界データの収集
第7回	Physical AIの実運用

本日

- 1 復習
- 2 推論時の工夫
- 3 システムの細部まで把握する
- 4 失敗を切り分ける
- 5 評価の設計
- 6 エンジニアリング環境と前半まとめ

今回のゴール

- **action chunk** にまつわる問題とその解決策を知る
- 実機で方策を動かすための **システム全体の見取り図**（制御スタック・デバッグ・評価）を持ち帰る

前提知識

- 第3回（模倣学習）の内容
- 第5回（VLAの基礎）の内容
- 深層学習

1

復習

復習：BC・VLA の学習と action chunk

2

推論時の工夫

3

システムの細部まで把握する

4

失敗を切り分ける

5

評価の設計

6

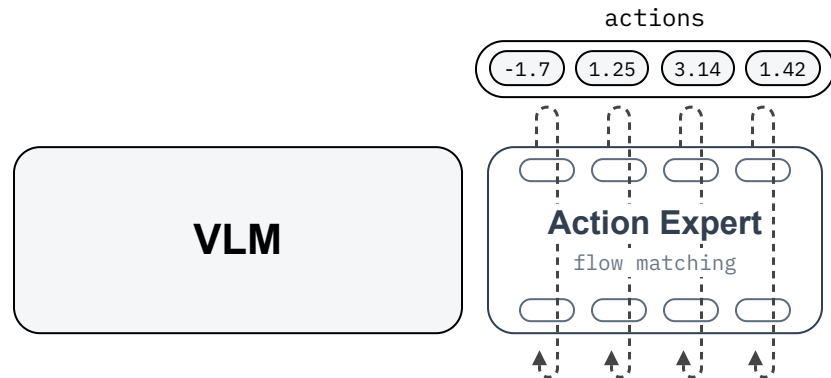
エンジニアリング環境と前半まとめ

- 専門家デモ $\mathcal{D} = \{(o, a^*)\}$ から、観測→行動の **方策** $\pi_\theta(a \mid o)$ を教師あり学習で学ぶ
- 単純な行動の直接 **MSE（二乗誤差）回帰** は、多峰なデモで **平均に潰れて破綻** する
- → 行動の **分布そのもの** を学習する生成モデル方策（**拡散モデル・flow matching**）が主流

$$\min_{\theta} \mathbb{E}_{(o, a^*) \sim \mathcal{D}} [\ell(\pi_\theta(o), a^*)]$$

詳細は第3回「Physical AIの学習（模倣学習）」を参照。今日は **この学習済み方策を実機で正しく動かす** ところから先の話。

- 事前学習済み VLM を backbone に、行動生成ヘッド (action expert) を接続
 - Web スケールの視覚・言語の知識を 行動生成に橋渡し。言語指示で条件付けできる
 - 行動ヘッドは **flow matching** などの生成モデル (例： π_0)
- そして BC も VLA も、実際には 1 ステップの行動ではなく **行動の列 = action chunk** を出力している

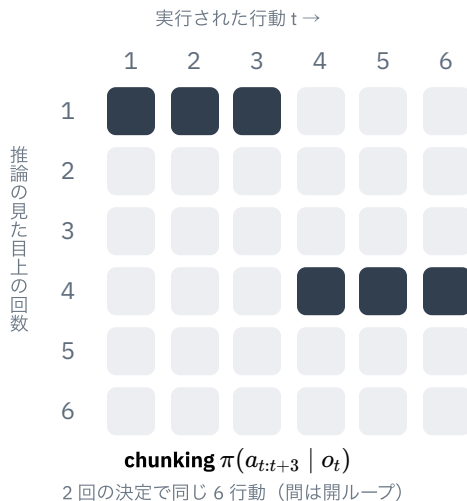


今日は、この action chunk 周りの問題と解決策を見ていく。

復習：action chunking、行動列をまとめて予測



- これまでは **1 ステップの観測から 1 ステップ分の行動だけ** を予測し、毎ステップ予測し直す **閉ループ方策** $\pi(a_t | o_t)$ 。例：制御が **50 Hz** なら **20 ms** ごとに予測をやり直す。
- **action chunking**：観測 o_t から k ステップ先までの行動の塊 $a_{t:t+k}$ を **まとめて予測** し、 k ステップごとに推論（間は再観測せず **開ループ** 実行）。



なぜまとめて予測？

- 1 実験的に **性能が大きく改善** することが多い。
- 2 k ステップに **1 回の推論** でよい
→ 1 回に使える **計算時間が増える**（重い生成モデルでも実機で回る）。
- 3 ぶれと **compounding error** が減り、動作が **滑らか**。

チャンク長 k のトレードオフ

- チャンク長 k は自分で決める ハイパーパラメータ
- 例：50 Hz で収集されたデータで $k = 32$ なら、 $20 \text{ ms} \times 32 \text{ steps} = 0.64 \text{ s}$ 先まで予測することになる

k が長すぎる ($k = 100$)



チャンク実行中は観測を見ない **open-loop 区間** が長い
 → 新しい観測への応答性が落ちる（外乱・動く物体に弱い）

k が短すぎる ($k = 2$)



境界のたびに再サンプル。色=サンプルされたモードが跳ぶ

チャンク境界のたびに分布から **別のモード** をサンプルする
 → **mode-jumping**・**jerky**（ガクガク）な動き

1 復習

2

推論時の工夫

非同期実行・チャンク接続・遅延補償（研究紹介）

3 システムの細部まで把握する

4 失敗を切り分ける

5 評価の設計

6 エンジニアリング環境と前半まとめ

- **同期実行**：チャンクを使い切ってから次を推論 → 推論が終わるまで **ロボットが停止** (stall / pause)
- **非同期実行**：現在のチャンクを実行 **しながら** 裏で次のチャンクを推論しておく

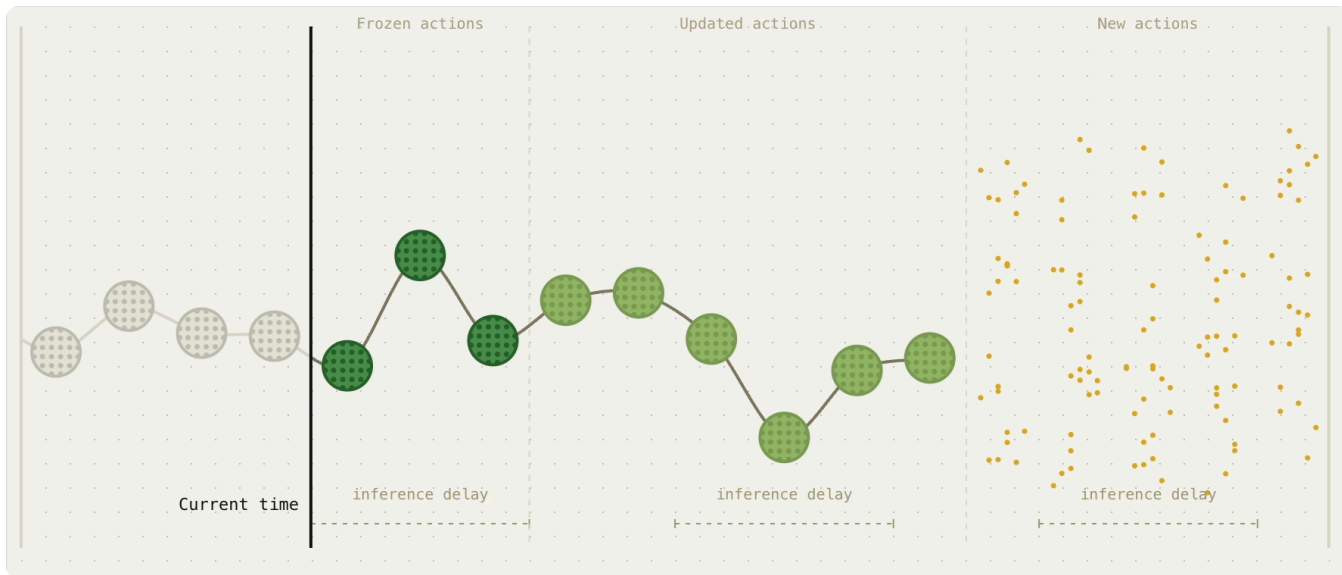


SmoIVLA [Shukor+, 2025]：行動キューの残りが閾値を下回ったら次チャンクを推論。

2つのチャンクを滑らかに繋ぐ ① — RTC (推論時)



- 非同期にしても、新チャンクが **別モード** だと境界で不連続が残る
- **RTC [Black+, 2025]** : 次チャンクの生成を **inpainting 問題** として解く — 必ず実行される冒頭は前チャンクの行動で **凍結**、残りは整合するよう **ガイダンス付きで生成**
- 拡散 / flow 系の VLA に **再学習なし** でそのまま適用できる

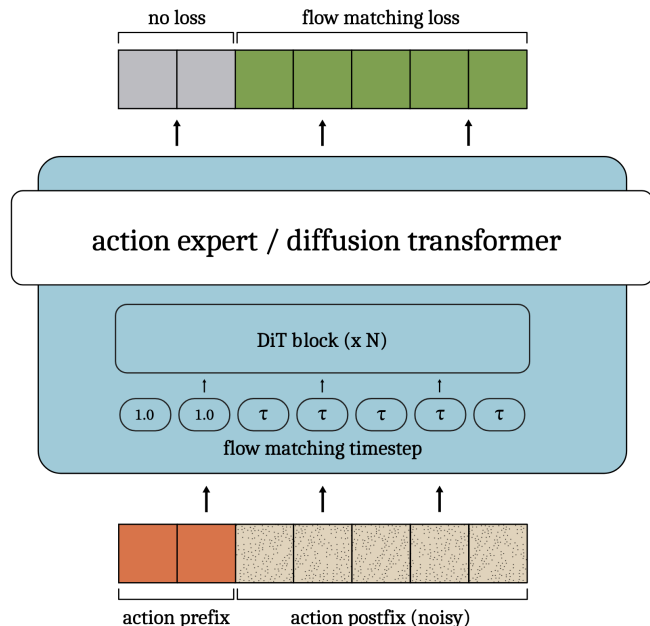


出典) [Black et al., "Real-Time Execution of Action Chunking Flow Policies" \(2025\)](#)

2つのチャンクを滑らかに繋ぐ ② — Training-time RTC



- 推論時 RTC は 推論時に追加の計算コスト が必要で重い → 同じ整合を 学習時に仕込む
- 実行済みプレフィックスに条件付けて続きを生成するよう訓練 (**diffusion forcing** の活用)
- 推論時の 追加コストはゼロ



```
chunk_horizon = 64, ttrtc_query_interval = 32
```

```
Chunk 1: |===executing(32)===|---remaining(32)---
```

```
↑  
trigger inference 2
```

```
|---d---|
```

```
↓ inference 2 complete
```

```
Chunk 2:
```

```
|prefix(d)|===newly generated===|
```

```
↑  
resume execution here
```

- 推論に使う観測は **推論開始時点** のもの → 新チャック実行開始時には **既に世界がずれている**
- VLASH** [Tang+, 2025] : 実行が確定している行動列を使って **実行時点 (将来) の状態を計算** ($s_3 = s_1 + a_1 + a_2$) し、**その状態に条件付け** て生成 → 速い反応と滑らかさを両立

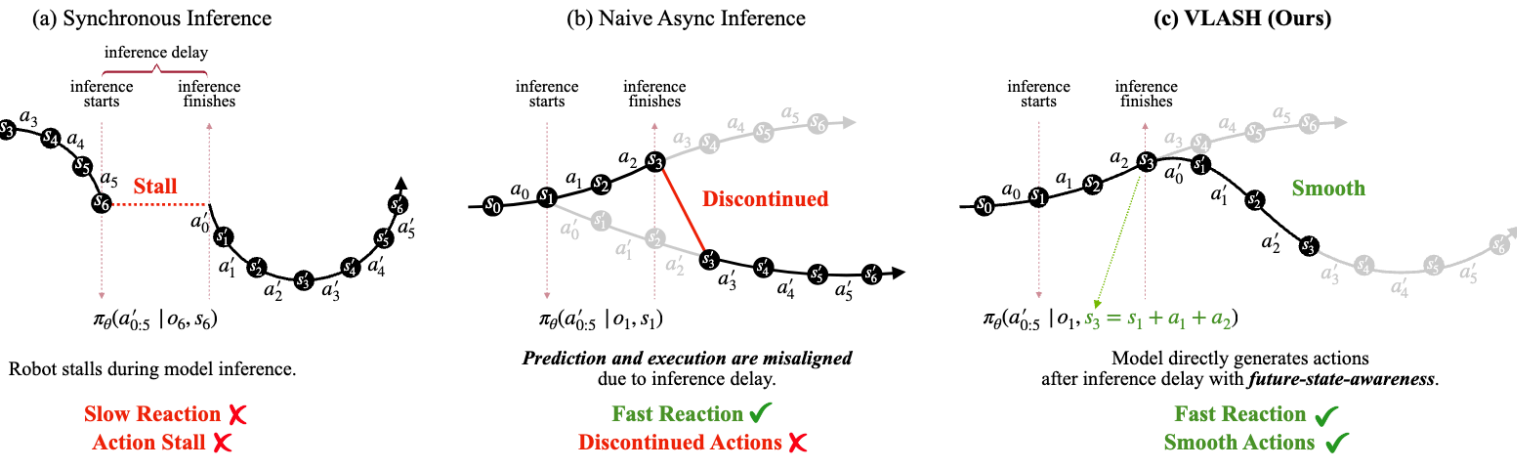


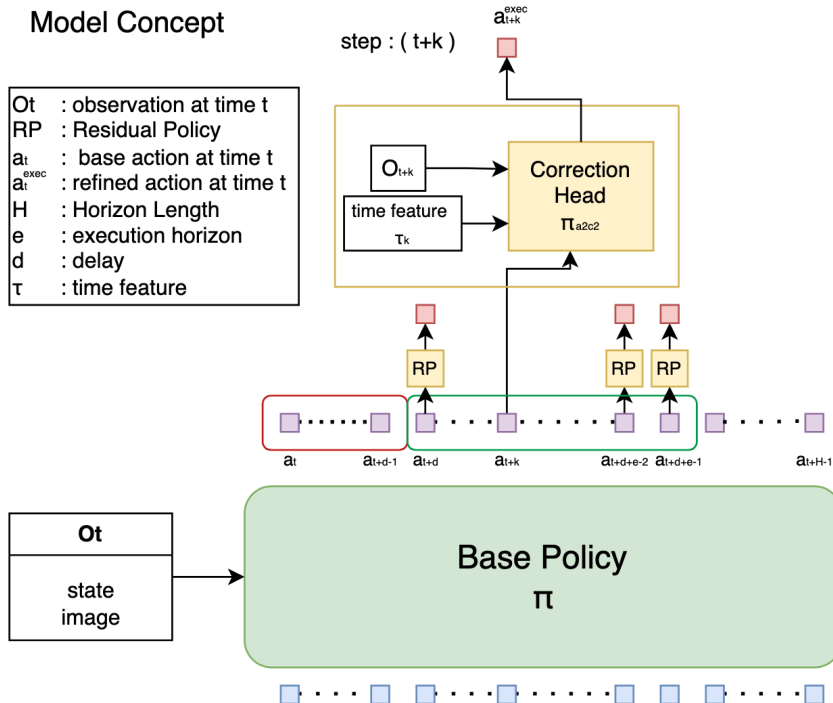
Fig. 3: **Comparison between VLASH and existing methods.** (a) Synchronous inference: the robot stalls during inference, introducing slow reactions. (b) Naive async: the model predicts based on stale state s_1 while execution begins at future state s_3 , causing misalignment and discontinuity. (c) VLASH rolls forward the robot state ($s_3 = s_1 + a_1 + a_2$) and conditions on the execution-time state, achieving fast reaction and smooth actions.

出典) Tang et al., "VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference" (2025)

観測のタイミングずれ ② — A2C2 (松尾研から)



- **A2C2 [Sendai+, 2025]** : 方策本体はそのまま、**軽量な残差ヘッド** を追加
- 最新の観測 + 実行中の行動を入力に、**高頻度で行動を補正** → ベース方策を選ばない



出典) Sendai et al., "Leave No Observation Behind: Real-time Correction for VLA Action Chunks" (2025)

1 復習

2 推論時の工夫

3

システムの細部まで把握する

制御スタック・観測経路・行動表現

4 失敗を切り分ける

5 評価の設計

6 エンジニアリング環境と前半まとめ

自分の動かしているシステムの 細部まで把握 せずに、正しく進めることはできない。

バー=1秒間の更新イメージ（下へいくほど速い）

VLA 方策 π

観測 $\rightarrow$ action chunk を生成（normalized $\rightarrow$ unnormalize）

~10 Hz

↓ 目標値（20 ms 間隔で 1 action ずつ）

Action 実行（制御周期）

チャンク内の各 action を順に目標値として適用

50 Hz

↓

ros2_control JointTrajectoryController

補間した軌道をアクチュエータへ送出

500 Hz

↓

Actuator 内部制御

電流（トルク）ループ、出力段は PWM 駆動

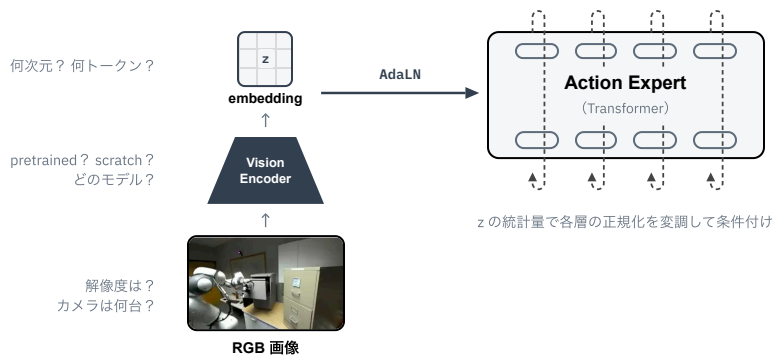
~10 kHz

1 周を数字で追う（例）

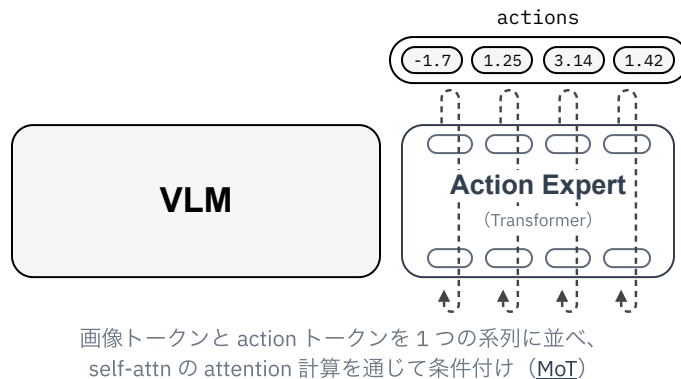
- 1 チャンク $32 \text{ step} \times 20 \text{ ms} = 0.64 \text{ s}$ を実行
- 2 実行し終わる頃に次の観測をモデルに query
- 3 推論は ~100 ms で完了し、次のチャンクが目標値に
- 4 episode が 1600 step なら、推論は合計 約 50 回

自分のシステムで、この数字を全部言えるか？

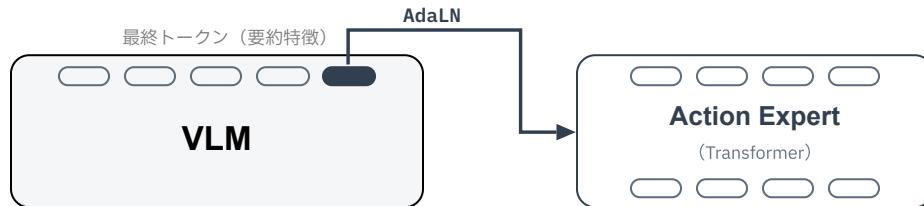
例①：AdaLN で各層に注入



例②：joint self-attn (attention 計算経路で条件付け)



例③：VLM の最終トークン (要約特徴) を AdaLN で注入



前処理 (リサイズ・クロップ・正規化) が 学習時と推論時で一致しているか も頻出の落とし穴。

- 方策が出す「行動」の定義が **1つずれているだけ** でロボットは暴れる。

座標系

end-effector pose ? **joint position** ? 手先か関節か、回転の表現は？

絶対 / 差分

絶対値 (abs) ? 前ステップからの **差分 (delta)** ?

正規化

どの **統計量 (mean / std, min / max)** で normalize ?
unnormalize は正しく対になっているか？

PRETRAINED との整合

fine-tune 元のモデルが学習した **データセットの行動形式** と、自分のデータの形式は一致しているか？

1 復習

2 推論時の工夫

3 システムの細部まで把握する

4

失敗を切り分ける

可視化ファーストとデバッグの順番

5 評価の設計

6 エンジニアリング環境と前半まとめ

- 直すには、まず **どの層の問題か** を切り分ける必要がある

① データ

デモの質・量、分布の偏り、収集時の設定ミス

② 学習

loss の挙動、過学習・未学習、ハイパーパラメータ

③ 推論

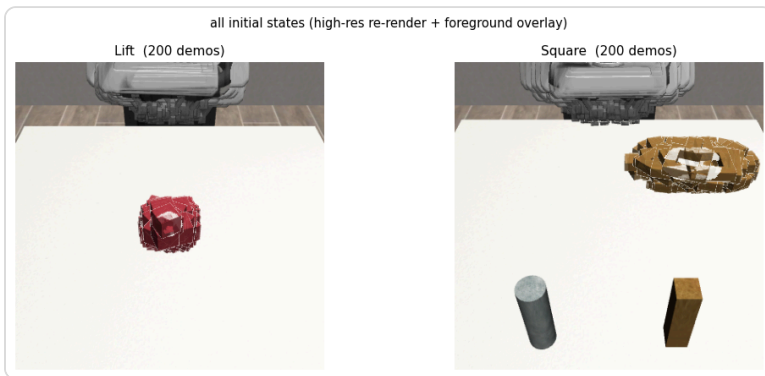
推論時の観測が **学習分布と整合** しているか（前処理・正規化のずれ）

④ 制御・HW

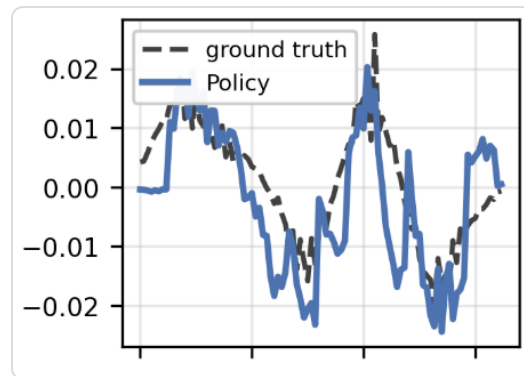
通信遅延、キャリブレーション、機械的トラブル

「なんか変」を「どの層の、何の問題っぽい」まで解像度を上げる。

- 1 初期状態画像の重ね合わせ：全エピソードの初期観測を重ねて、初期条件の分布を確認（左下図・第3回演習）
- 2 GT action vs policy action の重ね書き：val データで追従できているか（右下図）
- 3 モデルに入る直前の観測 をそのまま保存して目視する
- 4 chunk 境界の可視化：ガクつきが境界と同期していないか（次スライド）



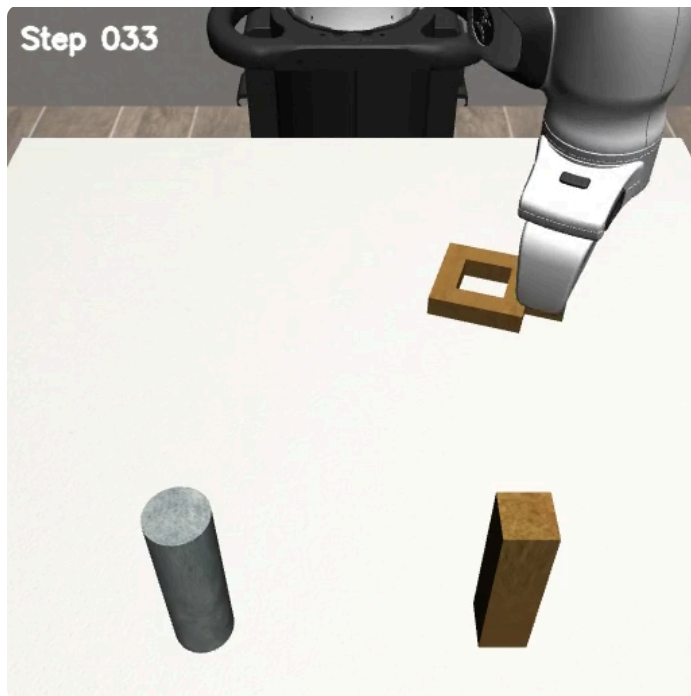
1. 初期状態の重ね合わせによる分布可視化（第3回演習より）



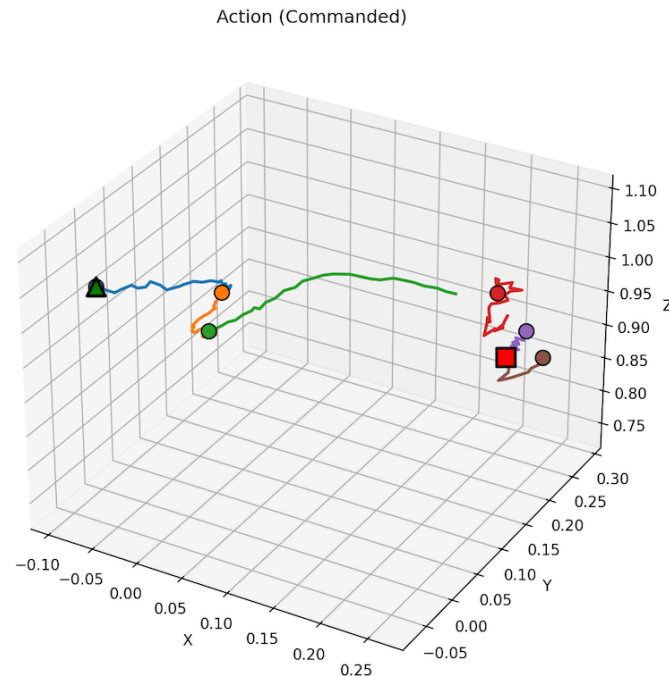
2. val データでの policy 出力 vs GT action の比較

可視化ファースト：chunk 境界の可視化

- コマンドされた action の軌跡を **chunk** ごとに色分け — 境界で軌跡が跳んでいないかが一目で分かる



同じ rollout の sim 上の実際の動き



action 軌跡の chunk ごとの色分け（色の切替=chunk 境界）

1

dataset の action を **open-loop** で replay

✓ うまくいけば **dataset** と **deploy** コードは問題なし

↓

2

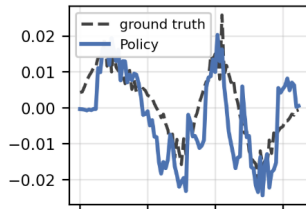
学習済みポリシー に差し替えて実機で実行

✓ うまくいけば **done**

↓ ダメなら

validation データの観測を policy に入力し、**GT action** と近い出力 が出るか検証

3



✓ うまくいくなら 実画像 → **policy** の経路が怪しい (カメラ・前処理)

または、validation では良さそうなのに実機でダメなだけ = **バグはなく性能が低いだけ** の可能性。その場合はデータと訓練設定の見直し：データ量に対して分布が広すぎ (多様すぎ) ないか？ データの多様性・量に対してパラメータ数は妥当か？

1 復習

2 推論時の工夫

3 システムの細部まで把握する

4 失敗を切り分ける

5 評価の設計
実機評価のプロトコルと記録

6 エンジニアリング環境と前半まとめ

試行回数が稼げない

初期条件の再現性がない

セットアップのドリフト

人の介在（リセット・介入）

評価者バイアス

分散はどこから来るか

- **seed**・**checkpoint**・**環境初期化** — 同じ手法でも数字は大きくぶれる
- シミュレーションでは固定プロトコルで大量に回して潰す。例：Diffusion Policy の robomimic 評価は **3 seeds × last 10 checkpoints × 50 inits**

初期条件

決められた範囲から **random** に選択（「なんとなく置く」をやめる）

A/B の条件を揃える

X の影響を知りたいときは **X 以外の条件をすべて揃える**

CHECKPOINT のぶれ

last X weights の **EMA** で高周波ノイズを取り除く／last X weights の **評価結果の平均** をとる

何を記録するか

成功率だけでなく **観測・行動・実現値・commit hash** を残す

記録があると失敗の解像度が上がる：

「なんか謎にガクガクしてるな」→「どうやら action chunk 間が滑らかに繋がってないっぽい」

A Careful Examination of Large Behavior Models for Multitask Dexterous Manipulation

TRI LBM Team

Abstract—Robot manipulation has seen tremendous progress in recent years, with imitation learning policies enabling successful performance of dexterous and hard-to-model tasks. Concurrently, scaling data and model size has led to the development of capable language and vision foundation models, motivating large-scale efforts to create general-purpose robot foundation models. While these models have garnered significant enthusiasm and investment, meaningful evaluation of real-world performance remains a challenge, limiting both the pace of development and inhibiting a nuanced understanding of current capabilities.

In this paper, we rigorously evaluate multitask robot

coordination, and offer the promise of producing general-purpose manipulation systems capable of performing arbitrary tasks.

Despite these strengths, single-task behavior-cloned policies remain brittle, exhibiting limited generalization to task variations or environments outside their training distributions. To overcome this brittleness, the field is increasingly adopting *Large Behavior Models* (LBMs) [2], [4]–[9]—visuomotor foundation models trained on large-scale multitask datasets containing action-level demonstrations. Inspired by the success of large-scale gener-

Toyota Research Institute の Large Behavior Model 論文は、評価の手続きそのものを主要な貢献として公開している

～2,700 実機ロールアウト (タスク×ポリシーあたり 50)

200 sim ロールアウト (タスク×ポリシー×条件あたり)

27% 人手評価に対して別人員が QA を実施

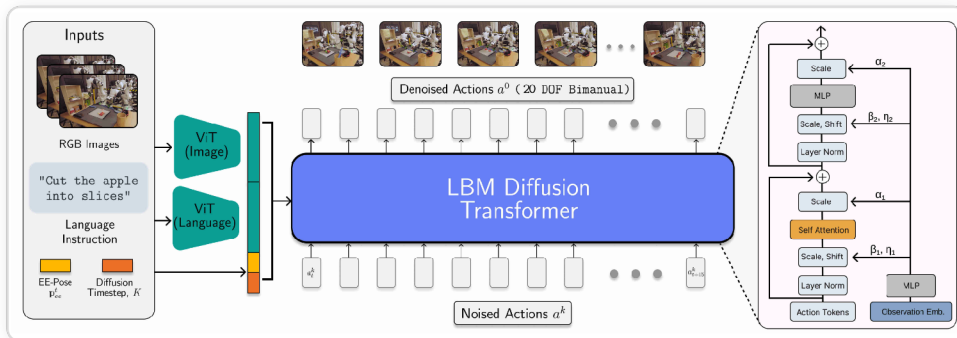
① 比較の公平性

② メトリクス

③ 統計処理

④ 誠実さ

ーの4つの観点から見る



出典) TRI LBM Team, "A Careful Examination of Large Behavior Models for Multitask Dexterous Manipulation" (2025)

BLIND

評価者はどのポリシーか知らない

順序

ポリシーの提示順をランダム化

初期条件

ポリシー間で**同一の初期条件**を共有

実機での実現法 — TEST BUNDLE (1 BUNDLE = 1 初期条件)

bundle 1 | 初期条件 #1



各ポリシー実行後に初期条件へリセット

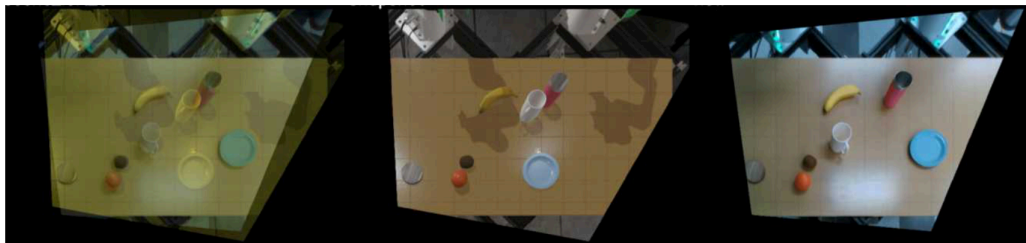
→

bundle 2 | 初期条件 #2



bundle ごとに順序を引き直す

→ ...



初期条件の再現：現在のカメラ画像と目標初期条件のオーバーレイ（左）を見ながら、作業者がシーン（中：目標、右：現在）を合わせる

TRI LBM 論文 Fig. 8 より。照明などの時間的ドリフトが特定ポリシーに偏らない

sim では単純：全ポリシーに**同一の sim パラメータと初期条件 (random seed)**。初期条件は学習データと同じ分布からサンプルし、**学習で使ったものは再利用しない**

成功率（SR）だけでは「あと少しで成功」と「何もしなかった」を区別できない → 進捗を測る **Task Completion (TC)** を併用

SIM — PREDICATE で自動判定

シミュレーション状態上に定義された**述語**（例：りんごの中心が bin 中心から閾値以内）の集合が**全て True なら成功**。同じ predicate で**部分成功・失敗モードの分析**もできる

実機 — RUBRIC (YES/NO 質問集)

進捗「掴んだか」と失敗「落としたか」を評価者が観察して回答。

TC = 達成マイルストーン数 ÷ 総数

rubric の回答は人為ミスを含むため、ラベルの誤差率を測って報告している

やったこと 評価者とは別の人員が全ロールアウトの約**27%**を再評価

測定結果 元の回答との不一致：成功判定 **2.31%**・rubric 質問全体 **6.25%**

VIOLIN PLOT

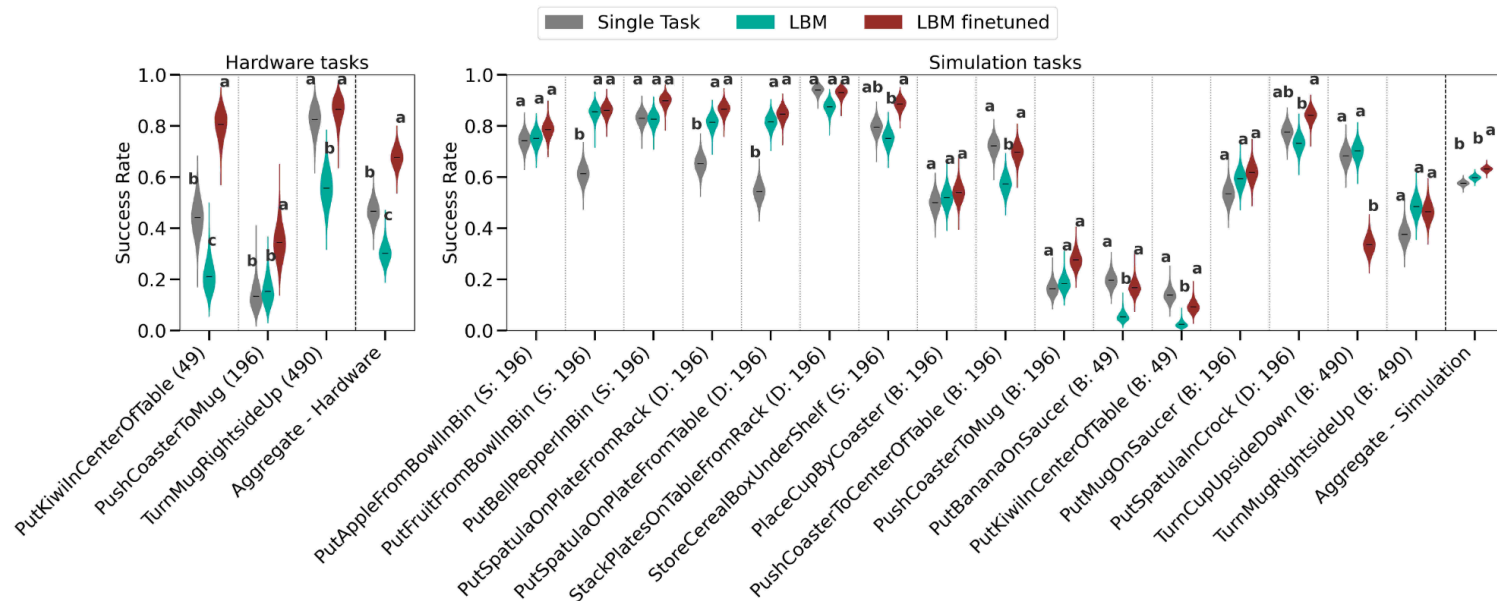
エラーバー1本ではなく、真の成功率 p の確からしさの分布 (posterior) を縦に描く。横に太い高さほど確からしい

なぜ CI (信頼区間) を使わないか

2つのエラーバーが重なっていても検定では分離できる場合がある。「重なる=有意差なし」の誤読を避ける

COMPACT LETTER DISPLAY (CLD) (a/b/c の文字)

ペアワイズ検定 + Bonferroni 補正の結果を要約。同じ文字を共有しない = 有意に分離



仮定の破れを明記する

- タスク横断の集計は **i.i.d. 仮定が破れる**と明記（試行数がタスクごとに事前固定でタスク上の一様分布からのランダムな抽出ではないため）
- タスク横断では **Type-I エラーは大域的に未制御**と明記
- 分布は各 checkpoint に対するものであり、**学習のランダム性 (seed) は捉えていない**と明記

異常値を放置せず分解する

- 成功率 0.335 の低さ → **200 中 89 回がタイムアウトまで初期姿勢のまま**と特定
- 実機 > sim の逆転 → 「**sim のタイムアウトが短すぎる？**」と仮説 → 実機を sim のタイムアウトで打ち切って再計算し確認
- 評価後に見つけた**正規化バグ**まで報告する

TRI ですら学習 seed の分散は取れていない。

何を測れていないかを把握する（論文なら書く）ことが、評価の信頼性をつくる

1 復習

2 推論時の工夫

3 システムの細部まで把握する

4 失敗を切り分ける

5 評価の設計

6

エンジニアリング環境と前半まとめ

pixi · git · コーディングエージェント

- **パッケージ管理**：依存ライブラリのバージョンを固定・記録し、同じ環境を誰でも作れるようにする
- **pixi**：conda + PyPI を一括で扱える宣言的なパッケージマネージャ

命令的 (PIP / CONDA を手で叩く)

- 実行する時と場所で結果が変わり、手順は記録に残らない → 再現不能
- conda と pip の混在は環境を壊す定番
- ロボット学習は **CUDA**・**ROS**・**sim** など非 Python 依存だらけで pip 単体では届かない

宣言的 (PIXI)

- **manifest**=意図 (pixi.toml)、**lockfile**=正確な真実 (全 OS 分のバージョン+ハッシュ)
- **conda + PyPI を 1 つの解決で扱う** (CUDA・ROS・faiss も)
- 環境はプロジェクト内に閉じる：**sudo 不要**・消せば消える・どのマシンでも同じ

```
$ pixi init && pixi add pytorch ros-humble-desktop --pypi lerobot # 依存を宣言
$ pixi install # lockfile どおりに完全再現 (別マシンでも同じ環境)
$ pixi run train # 実験コマンドも task として宣言 - 人も AI も同じ 1 コマンド
```

出典) M. Nishimura, “Portable & Reproducible Research Environments in the Age of AI Agents” (2026) speakerdeck.com/denkiwakame

- 基本的なことだが、バージョン管理は重要
- **git** を使ってプロジェクトを管理する

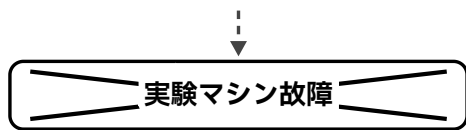
GIT のない世界

train.py

train_fix.py

train_fix_final.py

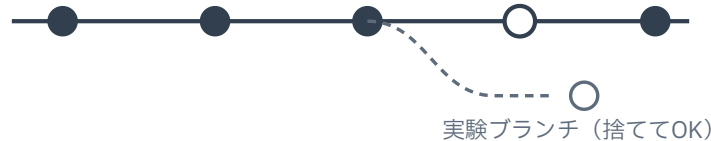
train_fix_final_本当に最終.py ← どれが動くやつ？



履歴もコードも**全部消えた**。「昨日動いた状態」には二度と戻れない

GIT のある世界

環境構築 ✓ 学習が回った ✓ **実機で動いた ✓** 壊れた変更 revert ✓



「動いた状態」をコミットで刻む → **いつでもそこに戻る**。push すればマシンが壊れてもコードは無事

ロボット実験は「昨日は動いたのに今日は動かない」が頻発する。**動いた状態に戻れることが**、そのまま切り分け（第4節）の武器になる。

- **Claude Code**・**Codex** など：端末を操作し、コードを書き、**実行して結果を確認するまで**を自律的に行う
- 人間の仕事は**タスクの分解**・**レビュー**・**フィードバック**に移る

ロボット学習での使いどころ

- 可視化スクリプト・データ変換など**周辺コードを丸ごと任せる**
(第4節の「まず可視化」が安くなる)
- ログ・エラーを読ませて**デバッグの仮説出し**
- worktree ごとに環境を立てて**実験を並列にばら撒く**

注意して使う

- エージェントは**指標を最適化する**, つまり評価をごまかす方向にも進み得る。**出力は必ず検証**
- **最小権限+使い捨て環境**で走らせる (sudo なし・壊しても作り直せる)

3 つは独立な道具ではない：pixi が使い捨ての再現環境を作り、git が履歴と並列化を支え、その上でエージェントが安全に暴れられる

1 推論時の工夫：action chunk のトレードオフを理解し、**滑らかさや反応性を確保**するための手法を理解する

2 システムの把握：制御スタック・観測の経路・行動表現などシステムの詳細を、具体的に把握する

3 デバッグ：いきなり実機で回さない。**層を切り分け、まず可視化**してから動かす

4 厳密な評価：初期条件の統制・ブラインド化・十分な試行数が本来必要、**何を測れていないかを把握**する

5 エンジニアリング環境：pixi・git・コーディングエージェントで**再現可能な実験基盤**を作る

「動くはず」ではなく、動くことを確認できる仕組みを作る